~ SY5 ~

Système d'exploitation

F$H

Rapport de projet

Achirafi Amal – Dahmane Ilan
Dominique Zoé

3ème année – Licence
Informatique générale

Université Paris Cité

Ce projet a été réalisé dans le cadre du cours de **Système d'exploitation** (**SY5**).

Le projet consiste à concevoir un *interpréteur de commande* (**shell**) interactif qui reprend les quelques fonctionnalités classiques des **shells** usuels : **FSH**.

Notre **shell** inclut des commandes internes et externes permettant d'interagir efficacement avec le système d'exploitation, il offre également plusieurs fonctionnalités intéressantes en plus de celles exigées par le cahier des charges.

Au niveau de l'implémentation, ce qui distingue notre projet est notre choix audacieux d'adopter une approche **orientée objet** afin de tirer pleinement parti des avantages de ce paradigme.

Les sections suivantes détailleront le processus de création en profondeur.

I) Architecture

A. Choix du paradigme

Pour réaliser l'implémentation de notre projet, il était nécessaire de faire un choix de conception efficace permettant de traiter les **commandes** saisies par l'utilisateur, les **pipelines**, et les **commandes structurées**, tout en assurant une bonne maintenabilité du code.

Étant donné notre immersion dans le paradigme orienté objet ces dernières années, nous étions parfaitement à l'aise avec les concepts, alors nous avons choisi d'utiliser (du moins de s'en approcher) le paradigme orienté objet dans notre conception du projet ?

Cela nous a permis de nous ramener à un environnement familier, où nous pouvions structurer le projet selon nos propres standards et appliquer certaines des techniques de conception vu en cours de **CPOO**.

D'autant plus que ce paradigme est tout simplement la manière la plus naturelle d'aborder le problème.

Malheureusement pour nous, le **langage C** n'est pas un langage nativement orienté objet, et il est un peu difficile d'implémenter certaines fonctionnalités fondamentales du paradigme que nous avions l'habitude d'avoir en **Java**.
Nous avons donc dû lui donner quelques petits coups de pouces, afin de nous rapprocher le plus possible du paradigme (ce qui a malheureusement un peu augmenté la complexité du code) et le peu que nous avions fait fut largement suffisant pour rendre le projet plus **modulaire** et **maintenable**.

B. Utilisation de classe et d'objet

Pour créer et manipuler différents types de commandes, nous utilisons des **classes** pour décrire le **type** de chaque **objet**.
Comme dit précédemment le **langage C** n'est pas orienté objet, donc la création d'une classe dans notre projet n'est pas explicite.

1. Classe

Dans notre projet, nous considérons de manière **implicite** qu'une classe est une **structure** (ce qui permet de faire plus ou moins de l'encapsulation) dont les champs sont les attributs de la classe.
Malheureusement le simple fait de créer un type **`A`** comme étant une structure pour dire que tout objet de type **`A`** est une instance de la classe n'est pas vraiment suffisant. Nous nous sommes d'ailleurs assez vites rendu compte des limites de cette approche.
En effet il n'y avait aucun moyen de réellement vérifier à l'exécution si un **objet** était l'instance d'une classe **`A`** (car il n'existe pas de méthode native en **C** comme **`instanceof`** en **Java**), ce qui pose problème car nous perdons là pas mal d'atouts très précieux comme **l'héritage** ou le **polymorphisme.**

Pour remédier à cela nous avons rajouté un champ supplémentaire dans chaque structure que nous considérons comme étant une **classe**, qui est un pointeur vers une autre structure appelée **`Class`** (pour classe de l'objet).

```
typedef struct _Class_ {

    /**
     * @brief Classe du parent. NULL si pas de parent.
     */
    const struct _Class_* const _super_;
    /**
     * @brief Le nom de la classe de l'objet.
     */
    const char* name;
    /**
     * @brief La taille d'allocation à la construction d'un objet.
     */
    const size_t size;
    /**
     * @brief Référence vers le constructeur d'une instance de la classe.
     *
     * Cette fonction sera appelée dynamiquement par la fonction `obj_new()` définie
     * dans `new.h`
     */
    void* (* const ctor) (void*, va_list*);
    /**
     * @brief Référence vers le destructeur d'une instance de la classe.
     *
     * Cette fonction sera appelée dynamiquement par la fonction `obj_destroy()` définie
     * dans `new.h`
     */
    void* (* const dtor) (void*);
    /**
     * @brief Référence vers la fonction `to_string` de la classe.
     *
     * Cette fonction sera appelée dynamiquement par la fonction `obj_to_string()` définie
     * dans `new.h`
     */
    char* (* const to_string) (const void*);
    /**
     * @brief Référence vers la fonction `equals` de la classe
     *
     * Cette fonction sera appelée dynamiquement par la fonction `obj_equals()` définie
     * dans `new.h`
     */
    int (* const equals) (const void*, const void*);

    /**
     * @brief Référence vers la fonction clone de la classe.
     *
     * Cette fonction est appelée dynamiquement par la fonction `obj_clone()` définie
     * dans `new.h`
     */
    void* (*const clone) (const void*);

    . . .


}* Class ;
```

Structure du type `Class` (Fichier `base/Class.h`**)**

Ainsi pour une classe donnée on crée une seule structure de type **_Class_** représentative de la classe, puis à chaque objet construit on fait pointer son champ de type **Class** (pointeur **_Class_***) vers la structure. De cette façon, pour savoir si un objet est l'instance (ou sous-type) d'une classe il suffit de vérifier si son champ de type **Class** (pointeur **_Class_***) pointe vers la structure, ou de remonter vers la classe parent et recommencer.

```
/**
 * @internal
 * Fonction qui vérifie récursivement si `_object_` ou un de ses ancètre
 * est `_class_`.
 */
static int obj_instanceof(const Class _object_, const Class _class_){

    if (_object_ == _class_)
        return 1;
    else if (_object_->_super_)
        return instanceof((const Class)(_object_->_super_), _class_);
    else
        return 0;

}
```

Fonction `obj_instanceof()` (Fichier `base/new.c`)

Comme on peut le voir dans la structure _Class_ précédente, le type ne renseigne pas seulement sur le l'appartenance d'un objet à une classe mais possède parmi ses champs pas mal d'éléments permettant d'interroger les instances d'une classe, comme par exemple : un constructeur (ctor) et un destructeur (dtor), une fonction de clonage (clone), etc. Toutes ces fonctions sont appelées dynamiquement à l'aide de la **liaison dynamique**.

Dans le fichier new.c il y a des fonctions comme `obj_new(void* __class__, ...)`, génériques, qui pour chaque objet donné en argument, appellent la vraie définition de la fonction donnée par la classe de l'objet. Cela permet de travailler sur des objets dont on ignore même la structure.

L'un des avantages majeurs dont nous avons pleinement bénéficié lors de l'implémentation du projet est l'utilisation de la **liaison dynamique**. Cela a grandement simplifié notre travail de conception.

Prenons un exemple concret **: la gestion des fuites de mémoire**. Dans notre conception, chaque objet est associé à une fonction de destruction qui lui est propre. Nous avons également imposé une contrainte stricte : tout objet dépourvu de destructeur ne peut pas être utilisé, car cela provoquerait un plantage du programme. Cette approche garantit que tous les objets sont correctement détruits.

Grâce à cette méthode, nous avons pu travailler sereinement avec les objets sans avoir à nous préoccuper manuellement de leur désallocation, y compris dans les cas où des allocations sont faites de manière récursive. Cela nous a permis de créer des compositions et des agrégations complexes, comme un **ArrayList** capable de contenir d'autres objets, eux-mêmes pouvant en contenir davantage. Ainsi, il devient possible de libérer tout un graphe de composition avec un

seul appel, rendant la gestion mémoire à la fois plus robuste et plus simple. Et cela a porté ses fruits car durant tout le déroulement du projet nous n'avons jamais souffert d'une fuite de mémoire.

(L'implémentation est en réalité un peu plus complexe que cela. Pour encapsuler les champs et les rendre privé de manière rigoureuse, nous avons utilisé une technique basée sur des **pointeurs opaques**, et les champs de la classe ont été implémentés à l'aide de macros **`#define`** pour permettre l'héritage.

Cependant ceci donne déjà l'idée générale de comment nous avons représenté les classes dans notre projet).

```
/**
 * @internal
 *
 * Fonction qui alloue et construit l'objet demandé.
 * Voir DOC. dans new.h
 */
void* obj_new(void* __class__, ...){

    // On cast en Class le pointeur (Class = struct _Class_* )
    Class _class_ = (Class) __class__;
    assert(_class_); // Vérification.
    // Allocation de la mémoire pour l'objet.
    void* object = calloc(1, _class_->size);
    assert(object); // Au cas où.

    // AJout de la classe dans l'objet.
    *(Class*) object = _class_;
    if (_class_->ctor){
        // Création de la liste d'arguments.
        va_list arguments;
        va_start(arguments, __class__);

        // Appelle du constructeur de l'objet.
        object = _class_->ctor(object,&arguments);
        va_end(arguments);

    return object;
}
```

Exemple de constructeur générique d'un objet (Fichier `base/new.c`)

2. Les types de base

Pour mener à bien notre projet et intégrer certains concepts issus des langages de haut niveau, nous avons dû nous doter d'outils adaptés, notamment des structures de données et des types d'objets plus avancés.

Notre modèle de classes nous a permis d'introduire un certain nombre d'objets familiers, dont nous avons conservé les noms habituels afin de faciliter la compréhension du code. Ces types incluent notamment des structures génériques comme les **ArrayList** et **LinkedList**, ainsi que des objets plus simples tels que **String**, **Integer** et **Character**.

Chacun de ces types a été doté de fonctions que nous avons optimisées autant que possible pour garantir des performances élevées et éviter de ralentir le programme. Cette approche nous a permis d'exploiter pleinement les avantages des abstractions offertes tout en maintenant une exécution efficace.

3. Le type Command

Le type **Command** représente la classe parent de toutes les commandes (y compris les commandes externes) dans notre shell. Il s'agit d'un type de base dont toutes les commandes héritent. Ce type est enrichi de nombreux champs et fonctions qui permettent de gérer efficacement les paramètres, les attributs des commandes, et les redirections. La classe est même muni de pas mal de fonctions qui facilitent le parsing du prompt.

Une méthode clé de cette classe est **`Command_execute(Command self)`**, qui doit être redéfinie par toutes les commandes héritant de ce type. Cette méthode, **polymorphe**, accepte n'importe quel objet appartenant à un sous-type de **Command** et exécute la fonction d'exécution propre à ce sous-type. Cela est rendu possible grâce à la **liaison dynamique**.

Ce mécanisme constitue un **pilier fondamental de notre architecture**, car il permet à de nombreux algorithmes de notre conception de fonctionner de manière **générique**. Par exemple, les **pipelines** et les **commandes structurées** ne sont en réalité que des **listes d'objets** de type **`Command`**, dont le type spécifique n'est résolu qu'au moment de l'exécution.

Cela nous offre une grande flexibilité : nous pouvons ajouter **autant de commandes** que nécessaire sans perturber l'implémentation existante. Et cela est rendu possible grâce à l'utilisation de fonctions **génériques**, qui garantissent une extensibilité tout en maintenant la cohérence du système.

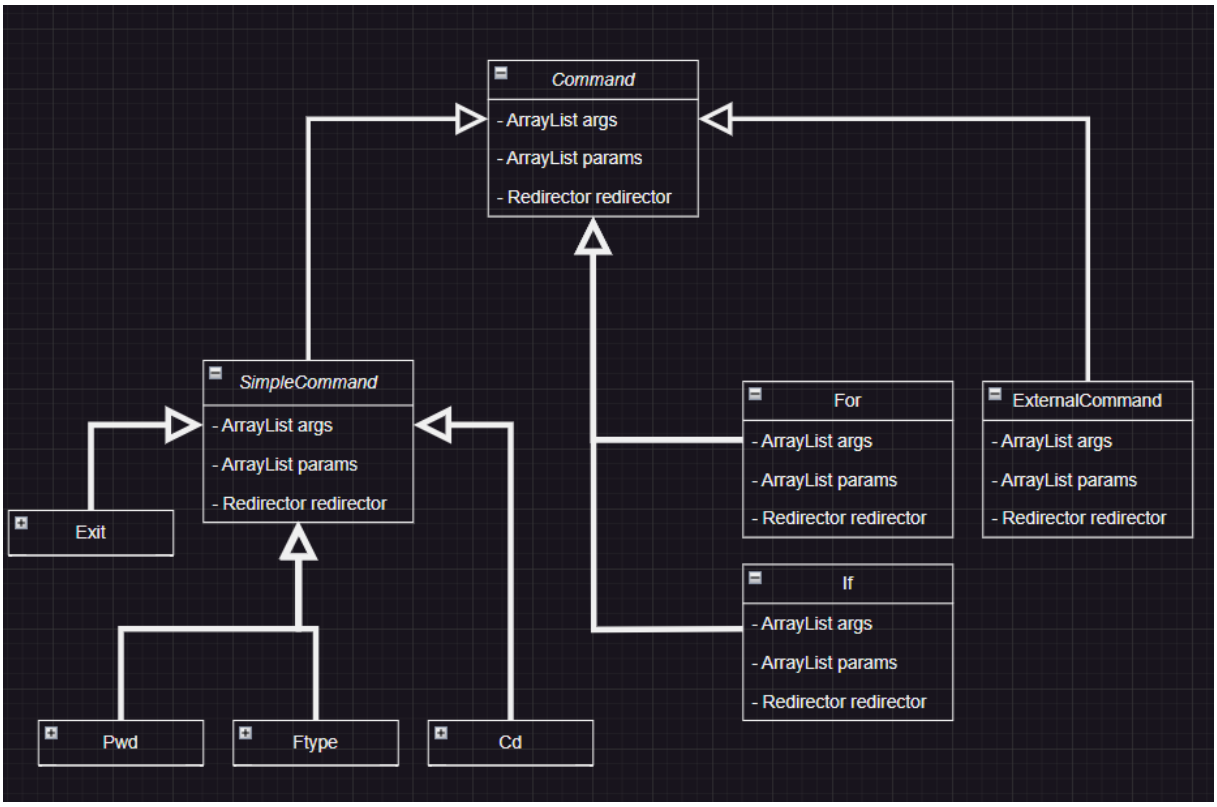
```c
    /**
     * @brief Fonction qui va exécuter la commande.
     *
     * Cette fonction est abstraite (déclanche une erreur si aucune redéfinition
     * n'est trouvée), à chaque appel la fonction va appeller la méthode `execute()`
     * de l'objet (liason dynamique). Cela va permettre plus tard de créer une
     * file de commandes toutes distinctes et de les éxécuter.
     *
     * @param self La commande à faire exécuter.
     * @return La valeur de retour de la commande après exécution.
     */
    int Command_execute(Command self);

/**
 * @brief Fonction qui vérifie si oui ou non le caractère actuellement pointé par
 * le pointeur vers le prompte de l'utilisateur est valide, c'est à dire
 * si on doit oui ou non arrêter la parsing de la commande.
 *
 * Un caractère est valide si et seulement si :
 *  - Ce n'est pas le caracètre null (fin de chaine).
 *  - Ce n'est pas l'un des caracètres '|', ';' ou '}'.
 *
 * Si le caractère n'est pas null (fin de chaine) et qu'il est entre guillemets (simple ou double) ou échapé
 * alors il est autamatiquement valide ('|', ';' et '}' seront considérés comme des caracètres normaux).
 *
 * @param quoted Booléen indiquant si le caracètre est entre guillemets simple.
 * @param double_quoted Booléen indiquant si le caracètre est entre guillemets doubles.
 * @param esc Booléen indiquant si le caracètre est echapé par \ .
 * @param prompt Le prompt en cours de paring.
 * @return Si oui ou non le caractère actuellement pointé par
 * le pointeur vers le prompte de l'utilisateur est valide.
 */
int Command_actual_char_is_legal(int quoted, int double_quoted, int esc, char** prompt);
```

Fichier `command/Command.h`

La classe **`SimpleComamnd`** est une classe enfant de la classe **`Command`** dont toute les commandes dites « simples » héritent. Elles sont considérées comme simples dans le sens où elles ne prennent que des arguments ou des attributs **booléens** (du genre **`-l`**, **`-a`**, … et non **`--out=out.txt`** ou **`-name poisson.txt`**) ou même rien du tout. Toutes les commandes internent de notre **shell** héritent de cette classe. Elles disposent d'un constructeur qui s'occupe directement de faire le parsing du reste du prompt jusqu'à une certaine condition d'arrêt. (Cette étape sera discutée plus loin dans le document.)

Les autres commandes étant plus complexe dans leur construction (**`For`** qui doit recevoir un répertoire à ouvrir, **`If`** qui doit vérifier l'existence d'un bloc **`else`**) sont des classes à part entières qui héritent directement de la classe mère (Command) pour profiter du **polymorphisme** et de la **liaison dynamique**.



Hierarchie des classes pour le type **`Command`**.

4. Le Shell

Contrairement à toutes les entités mentionnées dans ce document, le **shell** est le seul n'ayant pas été conçu comme un objet. En effet, le shell joue le rôle du code principal : il est lancé dès le démarrage du programme, ne possède qu'une seule instance, et reste actif jusqu'à la fin de l'exécution.

Au démarrage du shell, plusieurs opérations **préliminaires** sont effectuées. Par exemple, les structures ***Class*** des commandes sont créées. Ces structures, qui sont **immuables** et n'existent qu'en **une seule instance**, garantissent que toutes les commandes partagent un même type (et cela va de même pour tous les autres objets qui ne sont pas du type commande, il y a toujours un objet de référence unique qui permet de les identifier comme étant leur type).

Le prompt est également **formaté** conformément aux spécifications du cahier des charges. De plus, une table simulée pour gérer les **variables du shell** est initialisée. Bien qu'il ne s'agisse pas d'une véritable table (en raison des contraintes de temps on n'a pas pu écrire une structure de donnée digne de ce nom), elle en reproduit les fonctionnalités **essentielles**. Cette gestion des variables constitue l'une des fonctionnalités supplémentaires que nous avons intégrées dans le projet (voir détails plus loin dans ce document).

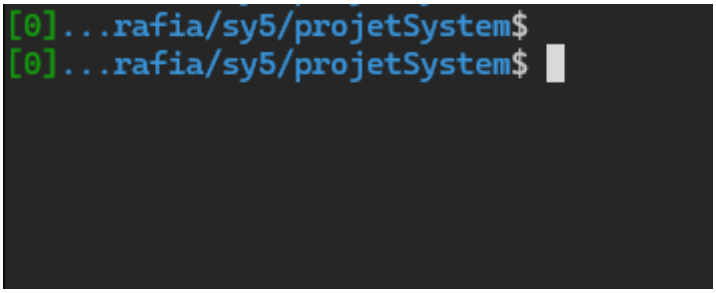
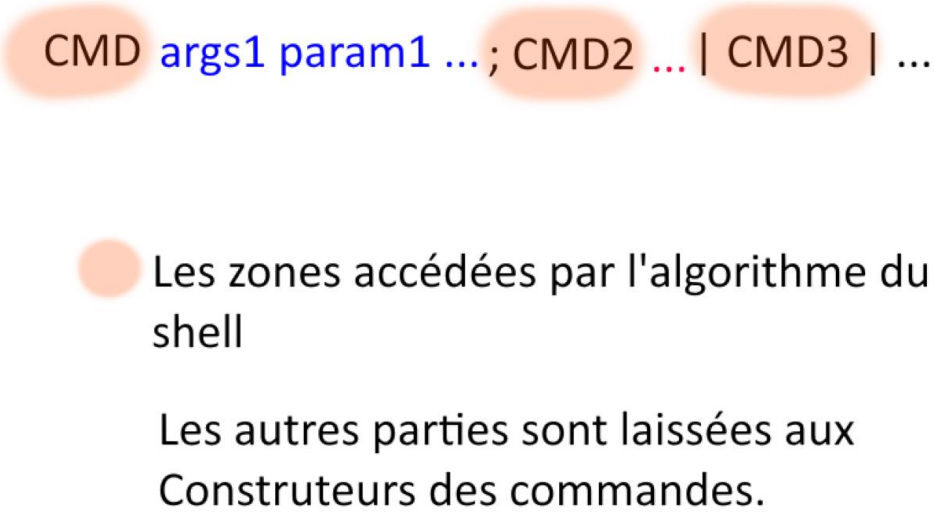
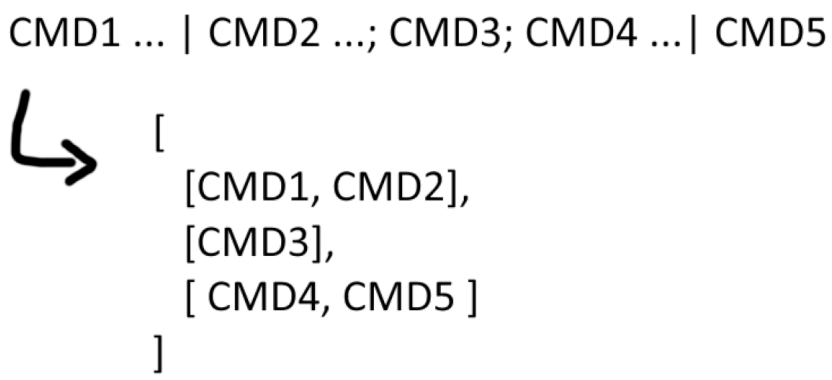


Image du shell après démarrage.

Lorsqu'une ou des commandes (structurées) sont saisies des fonctions vont se charger de faire le **parsing** de la chaine de caractère. L'algorithme utilisé pour faire ce traitement est très basique (point négatif de notre projet, cet algorithme est un peu brouillon et est un peu difficile à lire). Le principe reste tout de même assez simple : L'algorithme lit le nom de la commande, vérifie si elle existe et appelle **dynamiquement** le constructeur de l'objet **`Command`** pour se charger du reste de la chaine (choisir qui sont ses paramètres et qui sont ses arguments).
Nous avons fait en sorte que toutes les commandes travaillent sur la même référence du prompt de l'utilisateur (comme ça pas de copie, pas de fuite de mémoire), ainsi une fois que la commande a fini de se construire (ou pas dans cas l'algorithme s'arrête), le programme reprend la main là où le constructeur de la commande s'est arrêté et recommence sur le prochain mot qu'il lira.



Pendant l'exécution de l'algorithme tout caractère `;` indique la fin d'un pipeline. Les résultats de l'exécution de cet algorithme est une Liste de Listes dont chaque liste désigne un pipeline.



La fonction d'exécution du Shell ne va tout simplement parcourir cette liste et pour chacun des objets **`Command`**, appeler leur fonction **`Command_execute()`**. (Pour les pipelines, des redirections sont faites aussi.)

II) Prise en main

1. Commandes

Notre shell est capable d'exécuter toutes les commandes externes en tenant compte des variables d'environnement. Il intègre également des commandes internes spécifiées dans le cahier des charges.

Le format des commandes suit les standards des shells usuels, nous avons rajouté quelques flexibilités supplémentaires pour mieux se rapprocher des shells que l'on connait. Par exemple, les espaces délimitant les commandes ou séparant les blocs ne sont plus obligatoires. Ainsi, des commandes comme **`if true{echo a}else{echo b}`** ou **`if true { echo a } else { echo b }`** seront toutes deux acceptées sans problème.

Le shell dispose également de plusieurs mécanismes d'échappement des caractères. Par exemple, le caractère **`\`** permet d'échapper le caractère qui le suit. De plus, les guillemets (simples ou doubles) neutralisent les caractères spéciaux du shell, comme '**`;`**', '**`|`**', ou les **espaces**.

Ces fonctionnalités offrent une grande souplesse dans la rédaction et l'exécution des commandes.

2. Les redirections et redirection multiples

Notre shell propose des fonctionnalités de redirection, permettant la redirection des entrées ou des sorties (standard ou erreur) dans le cadre de commandes individuelles ou même au sein d'un pipeline.

L'option **`noclobber`** est activée par défaut et n'est pas modifiable. Ainsi pour rediriger la sortie d'une commande sur un fichier (inexistant) on utilise la caractère '**`> [fichier]`**' ou '**`x> [fichier]`**' avec **`x`** désignant le descripteur de la sortie à rediriger (**`1`** pour **`stdout`** et **`2`** pour **`stderr`**) et **`[fichier]`** la référence vers le fichier à écrire.

Pour écraser le contenu d'un fichier on peut contourner l'option noclobber en ajoutant le caractère **`|`** (exemple : **`ls >| a`** écrase le contenu de **`a`** s'il exsite).

Pour ajouter à la fin d'un fichier, on ajoute un autre caractère `>` (exemple : `ls >> a` ajoute à la fin du contenu de `a` s'il exsite).

Si un fichier n'existe pas au moment de la redirection de la sortie, il est créé.

On a également la même chose avec l'entrée standard, en utilisant cette fois-ci le caractère `<` comme '`< [fichier]`'. (Si le fichier n'existe pas il n'est pas créé et cela déclenchera une erreur).

Une fonctionnalité intéressante que nous avons ajoutée à notre shell est celle des **redirections multiples**.

La **redirection multiple** permet de rediriger une entrée ou une sortie (`stdout`, `stderr`, ou les deux) vers plusieurs fichiers **simultanément**. Cela évite non seulement de devoir dupliquer manuellement le fichier résultant, mais offre également la possibilité de rediriger le résultat d'une commande à un moment précis de son exécution, tout en préservant la sortie standard intacte. Cela s'avère particulièrement utile pour traiter des flux complexes dans un pipeline.

Voici comment cela se présente.

a) Redirection multiple en sortie

Pour activer la **redirection multiple** en sortie on ajoute un caractère supplémentaire à la syntaxe : le caractère `+`.
Par exemple : `ls >+ a >|+ b >>+ c >+d` écrit la sortie de la commande dans le fichier `a` (seulement s'il n'existe pas), rajoute également cette sortie dans `b` (en vidant son contenu avant), rajoute la sortie au contenu de `c` (à la fin), écrit la sortie de la commande dans le fichier `d` (seulement s'il n'existe pas) ***ET* écrit également la sortie sur la sortie standard**.
La raison pour laquelle on écrit aussi sur la sortie standard est que cela permet de ne pas fermer le descripteur de `stdout`. Ce qui permet par exemple d'utiliser les **redirections multiples** dans des **pipelines :**

`ls | sort >+ sorted.txt >>+ log.txt | grep test >+ test_sorted.txt | wc`

Le pipeline s'exécute normalement sans interruption, l'affichage de fin est fait et les redirections multiples sont également faites.

Si on veut tout de même ne rien afficher dans le terminal (ou rien écrire sur stdout) c'est-à-dire fermer complètement la sortie standard et rediriger le reste sur les fichiers, cela reste possible. Pour cela on doit ajouter le mot clé `>!` (cette redirection ne prend pas de fichier c'est juste un mot clé). Il indique au programme qu'on ne souhaite pas inclure `stdout`. Nous avons appelé ce procédé la redirection multiple **exclusive**.

Exemple : la commande `ls >+ a >|+ b >>+ c >+d >!` ou `ls >+ a >! >|+ b >>+ c >+d` n'écrira pas dans `stdout`, elle va par ailleurs le fermer complètement.

Ce mot-clé ne peut pas en revanche être mis avant les redirections. Cela déclenchera une erreur, il faut qu'il y ait au moins une **redirection multiple** déclarée pour la rendre **exclusive** !

Exemple : la commande `ls >! >+ a >|+ b >>+ c >+d` échoue alors que `ls >+ a >! >|+ b >>+ c >+d` n'échoue pas.

Lorsqu'une redirection (au moins) n'a pas de caractère `+` alors la redirection multiple est annulée. Par exemple `ls >+ a >|+ b >>+ c > d` redirigera tout sur le fichier `d` sans se soucier des autres fichiers ni même écrire sur `stdout` (qui sera donc fermé).

b) Redirection multiple en entrée

La redirection multiple en entrée fonctionne exactement comme celle en sortie, et suit les mêmes règles. La seule différence est au niveau de la syntaxe (`<` au lieu de `>`).

`wc <+ a <+ b ` lis en entrée le fichier `a` et le fichier `b` ***ET*** **lit également l'entrée standard**. La raison pour laquelle on écrit aussi sur la sortie standard est que cela permet de ne pas fermer le descripteur de `stdout`.

Ainsi le mot clé `<!` indique au programme qu'on ne souhaite pas inclure `stdin`, dans la lecture.

Ce qui est curieux ici c'est que contrairement à la **sortie standard** où l'on écrivait directement sur les fichiers, donc aucun problème, ici nous avons fait face à un problème majeur. Où écrire les données lues en entrées ? Le moyen le plus rapide et le plus simple était d'utiliser des **tubes anonymes** mais les tubes anonymes ont une taille **limite** (65 536 octets, modifiable mais présente toujours des risques de blocages), ce qui peut conduire à des **auto-blocage** si les fichiers sont trop **grands** (cela est arrivé) c'est pour cela que nous nous sommes assurés que la lecture des fichiers en entrée dans une **redirection multiple** soit faite en **parallèle** sur un autre processus enfant, ce qui permet lors d'un blocage d'attendre que le moment où le processus **principale** (qui lui va continuer) va lire l'entrée pour vider libérer la place (et ça marche).

3. Variables du shell

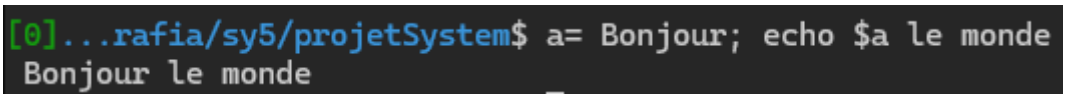
Dans notre shell nous avons ajouté une fonctionnalité permettant de créer des **variables du shell**. Il s'agit d'une valeur que l'on stocke dans la **table des variables du shell** et que l'on identifie à l'aide d'un identificateur (un nom commençant par un caractère `$`).

C'est ce mécanisme que nous utilisons dans les boucles **for**, les variables de la boucle sont en réalité des variables du shell, « locales » (locale car elles sont détruites par la commande après exécution) et c'est la boucle qui met à jour les valeurs de la variable. Cela implique donc que l'on peut donner n'importe quel nom à la variable de la boucle for (donc on ne se limite plus à une lettre de l'alphabet).

Ces variables peuvent être utilisées dans n'importe quelle commande (même en dehors de `for`). Les commandes sont implémentées de manière à reconnaitre si un de leur argument est une variable et elles cherchent la valeur dans la table des variables du shell (ET même dans les variables d'environnement, ce qui signifie que `echo $HOME` affiche la référence absolue vers le répertoire `~`).

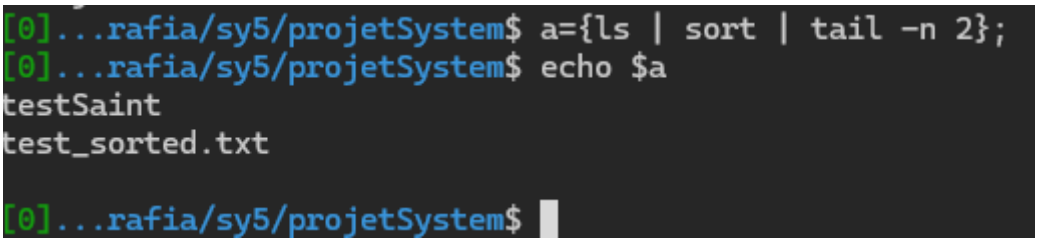
La syntaxe pour créer une variable est : `nom= valeur `. La seule contrainte ici c'est que le '`égale`' reste collé au nom de la variable que l'on veut créer (sinon le shell considérera `nom` comme étant le nom d'une commande et va chercher son constructeur). Après '`égale`' tout ce qui vient après est donné à la variable comme valeur, la fin du prompt ou les caractères ' `;`', '`}`' et '`|`' marquent la fin de la valeur.

Exemple : `a= Bonjour; echo $a le monde` affichera 'Bonjour le monde'.



La durée de vie des variables est la même que celle du Shell (les valeurs sont donc perdu au redémarrage)

On peut également affecter à la variable, la sortie d'une commande (un peu comme rediriger **`stdout`** dans une variable) en écrivant la commande entre accolade après le **`égale`**.



III) Conclusion

En résumé, notre shell se distingue par sa flexibilité, son extensibilité, et ses nombreuses fonctionnalités avancées. Grâce à une conception orientée objet, nous avons pu intégrer des mécanismes comme la **liaison dynamique**, des commandes **polymorphes**, et des structures génériques, garantissant une implémentation robuste et évolutive. Les fonctionnalités de redirection, y compris les redirections multiples une grande liberté dans l'exécution des commandes.

Malgré les contraintes de temps, nous avons respecté les spécifications du cahier des charges tout en ajoutant des améliorations significatives. Ces choix de conception permettent non seulement de répondre aux besoins immédiats du projet, mais également de poser des bases solides pour de futures extensions. Ce shell constitue ainsi une solution pratique et performante, alliant simplicité d'utilisation et richesse fonctionnelle.

Bibliothèques utilisées

- La bibliothèque Readline, pour les prompts.
- Doxygen pour la génération de la documentation HTML à partir des commentaires.